RescueMind：現地で「目で確かめてから」届ける災害物資ロボット

ローカルVLM（Qwen2.5-VL-3B / V100）× ROS 2 Jazzy × Gazebo Harmonic ― 判断を誤らないための検証可能な自律移動

項目	内容
提出者	omnicampus アカウント：tetsutetsu
コード	https://github.com/fe2131-art/rescue-mind
動画	YouTube（タイトル：PAI最終課題_tetsutetsu）。走行区間は2〜3倍速、AI判断区間は等速（倍速は動画内に常時表示）
使用ツール	ROS 2 Jazzy / Gazebo Harmonic（Docker, EGLヘッドレス描画） / PyTorch 2.5.1 + Transformers 4.53 / Tesla V100 32GB
AIの活用	①自然言語指示の分類（TF-IDF＋ロジスティック回帰）②搭載カメラ画像のローカルVLM推論（閉集合対数確率スコアリング）
主結果	事前登録した30試行（自律走行＋実撮影＋実推論）で VLMの許可／拒否判断 30/30 正解（誤許可 0 件・誤拒否 0 件）、Gazebo真値・ROS bag による独立監査まで合格 28/30。VLM判断遅延の中央値 2.79 秒

1. 課題の概要と目的

災害時の物資配送では「在庫表では水があるはずの棚に、実際は救急箱が置かれていた／空だった」という情報と現物の食い違いが起こりやすい。指示どおりに機械的に運ぶロボットは、誤った物資を自信満々に届けてしまう。本作品は、自然言語で依頼を受けた差動二輪ロボットが物資拠点まで自律移動し、その場で静止して搭載カメラで現物を確認、ローカルVLMが「依頼された物資がある」と高い確信で一致した場合にだけ避難所Bへ向かい、違えばその場で停止して人に判断を委ねるタスクを実装した。

狙いは「AIに運ばせる」ことではなく、AIの判断が間違っているときにロボットが動かないことを、ログと物理シミュレータの真値で検証可能にすることである。ロボットは物資の把持・運搬は行わない（現物確認と避難所への接近までを対象とし、配達完了とは表現しない）。

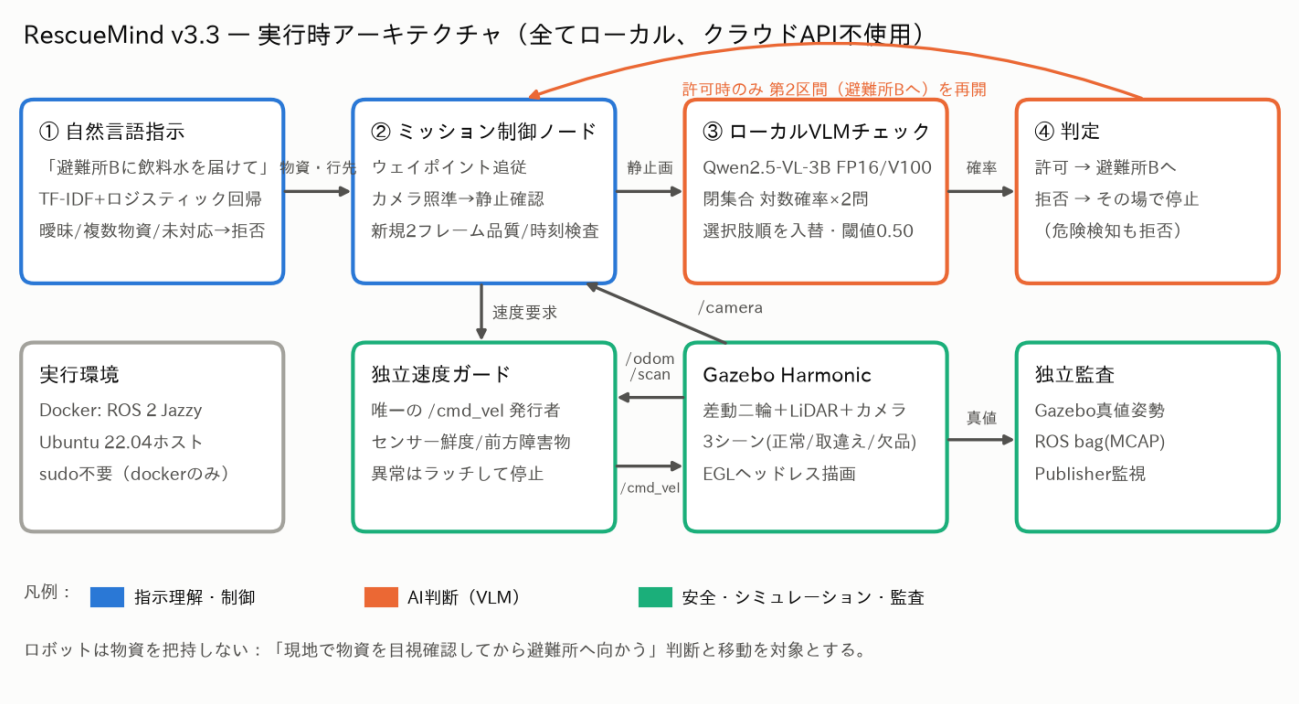


図1 実行時アーキテクチャ。全推論はローカルGPUで実行し、クラウドAPIは使用しない。

2. AIの活用と工夫

2.1 現地確認VLM：自由生成から「閉集合の対数確率スコアリング」へ

引継ぎ版（v3.2）は VLM に JSON を自由生成させていたが、実際の Gazebo 画像で評価すると、質問2はほぼ常に「なし」、質問1は空の棚で「水」と幻覚し、96応答中21件は解析不能だった。v3.3 では各質問を3択の選択式にし、生成はせず次トークンの選択肢文字（A/B/C）の対数確率を1回の順伝播で読み取り softmax で確率化する。2問は選択肢の順序を入れ替え、文字位置バイアスで両問が同じ誤答に一致することを防ぐ。危険（火災・煙・瓦礫）は別の Yes/No 質問で判定する。許可条件は「2問とも依頼物資が最大かつ確率 0.50 以上、危険なし」。

方式（開発セット48枚, τ=0.50）	正しい判定	誤許可	誤拒否	解析不能	1問あたり遅延
v3.2 引継ぎ：自由生成JSON（Qwen 3B）	37/48	0	11	21	1.98 秒
SmolVLM2-2.2B：対数確率（2モデル合議の候補）	32/48	0	16	0	1.34 秒
v3.3 採用：対数確率（Qwen 3B）	48/48	0	0	0	0.70 秒

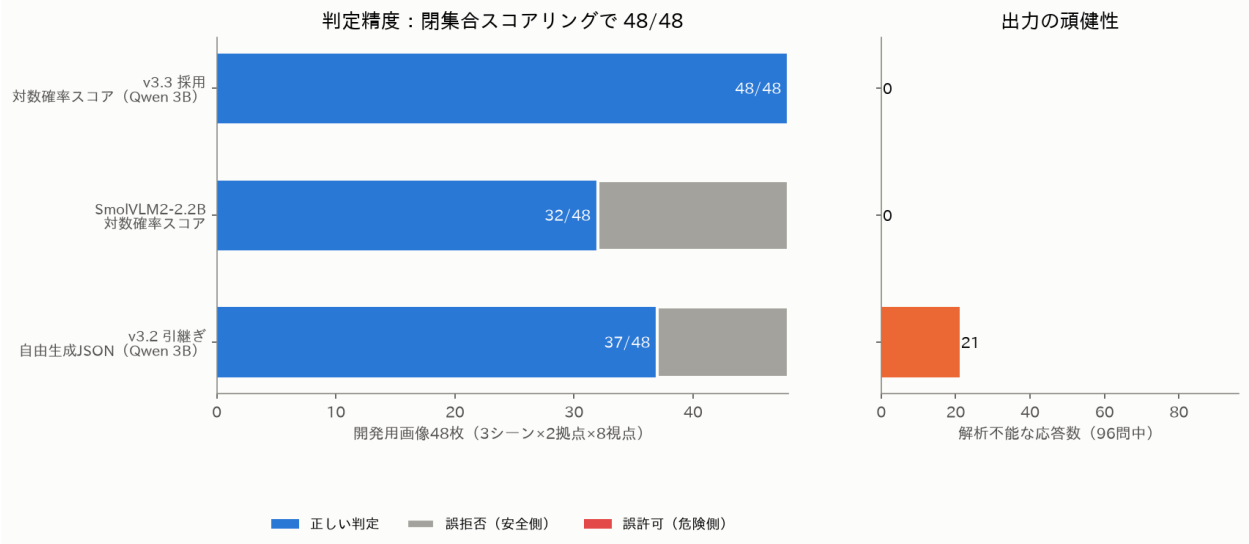


図2 開発セット（テレポートで視点をランダム化した48枚）での方式比較。閾値はここだけで決定した。

2.2 開発セットと評価の分離（事前登録）

プロンプトと閾値は、評価とは別に作った開発セット（3シーン×2拠点×8視点、seed=7）だけで決めた。真の陽性の最小確率 0.569、陰性の最大確率 0.019 と大きく分離しており、閾値 0.50 を experiments/PREREGISTRATION_v33.md に固定してから、事前に順序・ID まで決めた30試行の閉ループ評価を一度だけ実行した。

2.3 自然言語指示

文字 n-gram の TF-IDF＋ロジスティック回帰で「水／医療物資」を分類し、避難所名を正規表現で抽出する。両方の物資が含まれる・どちらも含まれず確信度が低い・未対応の避難所、のいずれも出発前に拒否する（既定値で黙って選ばない）。

3. ロボット制御と安全設計

- 2区間ミッション：物資拠点へウェイポイント追従 → 拠点中心へカメラを向けて静止 → オドメトリで静止を確認してから新規フレームを2枚取得（画質・ROS時刻・シミュレーション時計を検査）→ VLM判断 → 許可時のみ避難所Bへ。推論中・拒否後もゼロ速度と静止を監視し続ける。
- 独立速度ガード：ミッションは要求トピックにしか出力できず、/cmd_vel の唯一の発行者は別プロセスのガード。LiDAR前方距離・カメラ/LiDAR/オドメトリの鮮度・速度上限・後退禁止を10 Hzで判定し、異常はラッチしてゼロ速度。
- Publisher監視：DDSグラフを200 msごとに監視し、/cmd_vel に第2の発行者が現れたら非常停止。

- 証拠に基づく完了判定：成功は「許可→避難所到着」をミッション自身の記録ではなく、独立レコーダが購読した Gazebo真値姿勢・ROS bag（MCAP）・時系列（許可より前に避難所へ近づいていないこと）で監査して初めて COMPLETED。拒否は「拒否＋2.5秒停止保持＋真値で拠点から動いていない」で初めて COMPLETED_EXPECTED_DENIAL（VLMのエラー停止は拒否成功に数えない）。
- 環境：Ubuntu 22.04 ホストで、Docker（ROS 2 Jazzy + Gazebo Harmonic + CUDA PyTorch）を sudo なしで実行。Gazebo は EGL ヘッドレス描画（V100）で、カメラと gpu_lidar を描画する。

4. 実験結果：事前登録30試行の閉ループ評価

3シーン（正常配置／水と救急箱の取り違え／欠品）×依頼物資2種×5反復を、試行ごとに Gazebo・ブリッジ・ガード・VLM を新規起動して実行した。同一ワールドの反復であり30個の独立なシーンではない点に注意。VLMの判断（許可／拒否）は判断に到達した 30 試行中 30 試行で期待どおり、危険側の誤り（誤許可）0 件。判断に加えて Gazebo 真値・ROS bag・時系列の独立監査まで全て合格した試行は 28/30（不合格の内訳：拒否は正しいが停止証拠の時刻整合が不合格 1件、オドメトリ誤差が監査上限0.25 mを超過（真値監査） 1件）。VLM判断遅延（静止画取得→判断）の中央値 2.79 秒（最大 3.48 秒）、1試行の全体時間の中央値 66 秒（起動・準備確認を含む）。VLM判断前のインフラ失敗による再試行は 1 件（台帳に記録）。

※ 評価Cの前に2回の評価を中断した（台帳はいずれも公開）。評価A（9試行・12回実行）は、起動直後のミッションノードが3秒以内に /odom・/scan を受信できないDDS探索の競合で 4 回失敗（VLM判断に至った 8 回はすべて正解 8/8）。評価B（5試行・9回実行）は、Aの中断時に監督プロセスを強制終了したため孤立したスタック（速度ガードを含む）が残存し、Publisher監視が「/cmd_vel の発行者が2つ」を正しく検知して全 9 回を判断前に停止した。起動待機・停止シグナル・孤立スタック検出を修正し（判断ロジック・閾値は不変）、30試行すべてを評価Cとして最初からやり直した。以下はCの結果。

シーン	期待	許可→避難所到着	拒否→拠点で停止	拒否（証拠不足）	失敗
正常配置	許可	9	0	0	1
取り違え	拒否	0	9	1	0
欠品	拒否	0	10	0	0

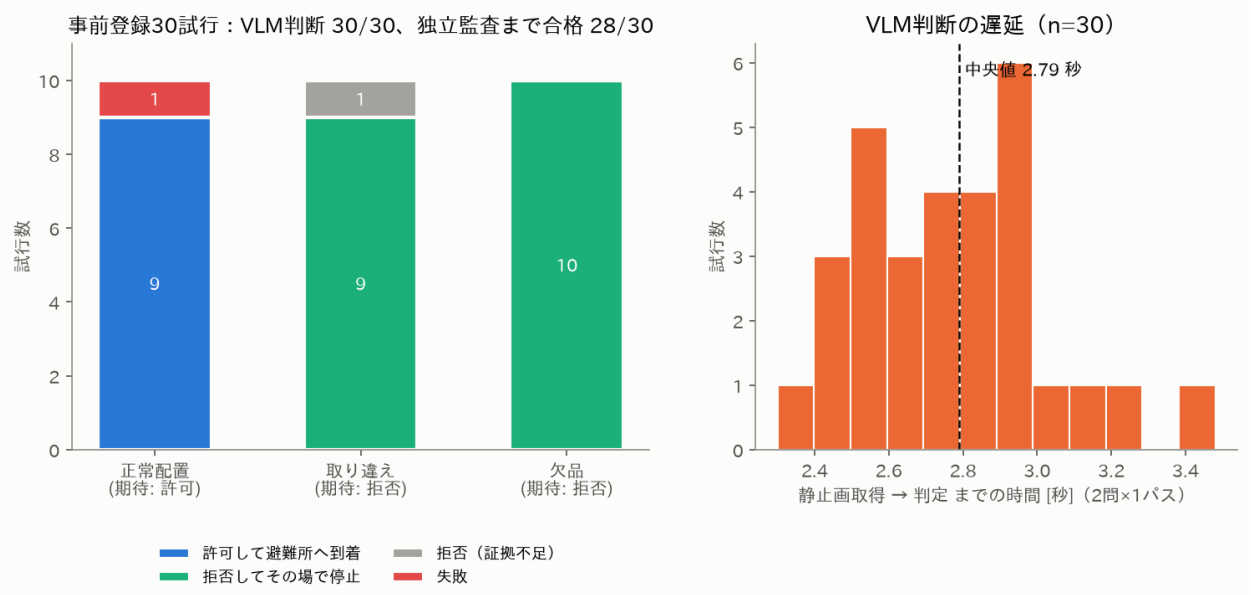


図3 30試行の結果（左）とVLM判断遅延の分布（右）。

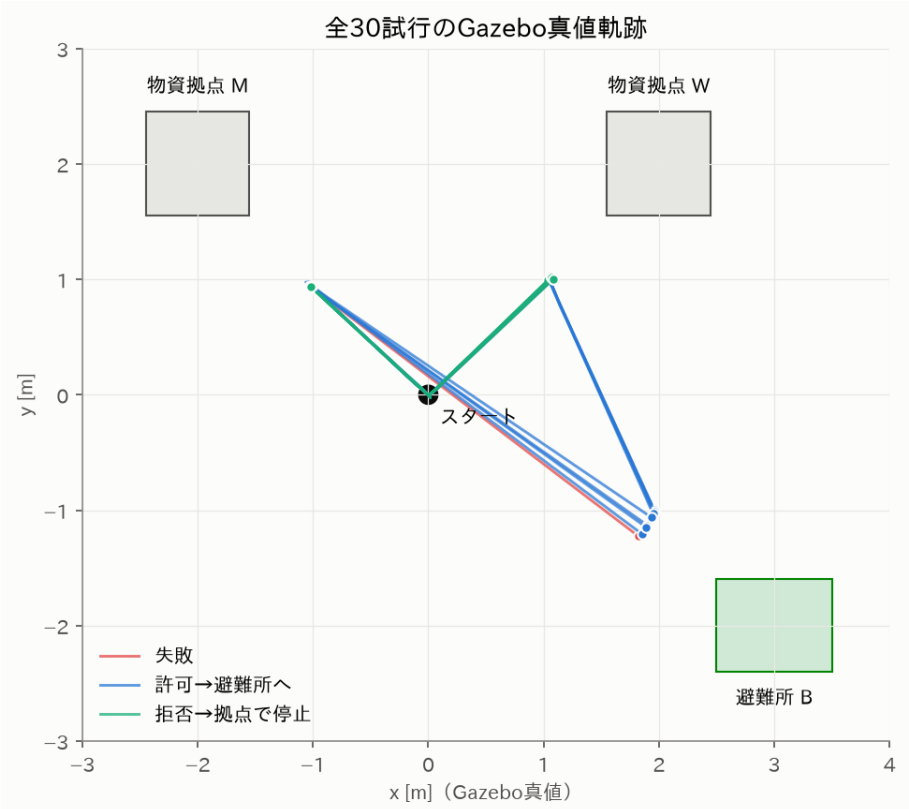


図4 全試行の Gazebo 真値軌跡。許可された試行だけが避難所Bへ向かい、拒否試行は拠点で停止している。

5. 実際に動かして初めて分かった問題と対策

引継ぎ時点のコードは506件の単体テストに合格していたが、実 Gazebo・実 ROS 2・実 GPU では一度も動いていなかった。実行して次の問題を発見し、すべて再現→修正→確認した（詳細は docs/FINDINGS.md）。

症状	原因	対策
オドメトリ上は前進1.5 mだが、実車体は横に滑り傾いていた	ロール済み車輪リンクの子座標系で関節軸を指定（車輪が鉛直軸回転）	軸をモデル座標系で指定。Gazebo真値で監査する仕組みを追加
オドメトリ基準の「独立」監査が不正走行を見逃し得た	制御器と監査器が同じ車輪オドメトリを信用	OdometryPublisher の真値を別トピックで監査（制御器は購読しない）
走行開始2秒でカメラ途絶判定	Best Effort QoS で0.9 MB画像を断片ロス（102枚中10枚のみ到達）	画像購読を Reliable/KEEP_LAST(2) に統一
Publisher監視が起動直後に誤停止	DDSのノード名メタデータ到着前の一時的な不明表示	アーム前・猶予内・1件・型一致のみ保留扱い（テスト追加）
物資が画面外／LiDARが自機マストを検出	カメラが低すぎる／gpu_lidar は visual を描画して測距	カメラを0.62 mのマストへ。visibility flag/mask で自機装飾を除外
VLMが水ボトルを認識しない	小道具が単なる水色円柱（Sim-to-VLMギャップ）	半透明PET・肩・首・キャップ・ラベルの6本パックに作り替え
評価Aで起動直後に /odom・/scan 未受信	新規ノードのDDS探索が3秒の固定待機に間に合わない	全必須ストリームの初回受信まで最大20秒待機（ゼロ速度）
評価Bで「/cmd_vel 発行者が2つ」と全停止	中断時のSIGKILLで速度ガードを含むスタックが孤立・残存	段階的停止（SIGINT→SIGTERM→SIGKILL）、docker --init、起動前の孤立スタック検出

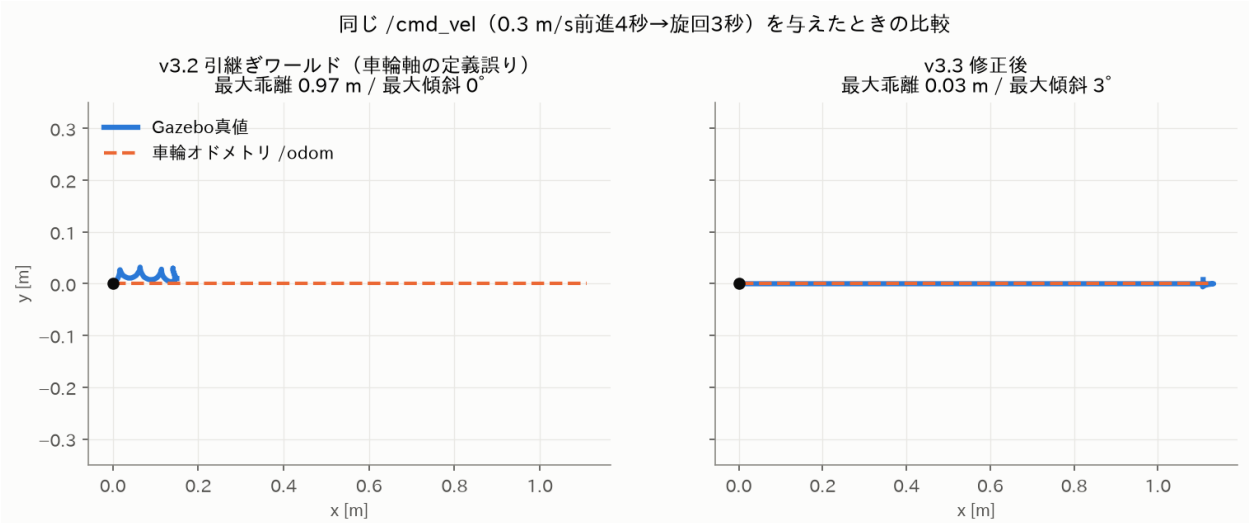


図5 同じ速度指令に対する車輪オドメトリとGazebo真値。左：引継ぎ版の軸定義（車輪は回るが車体はほぼ進まず、オドメトリだけが1.1 m進む）、右：修正後。

6. 実用性・応用可能性

- 人と協調する停止：誤物資・欠品・危険を検知すると「出発しない」ことが既定動作で、現場の人に確認を委ねる。物流倉庫・病院内搬送・避難所運営など、在庫情報と現物がずれる現場に直接対応する。
- オフライン・オンデバイス：通信が途絶しやすい災害時を想定し、推論はすべて単一GPU上で完結（クラウドAPIキー不要）。3Bモデル・FP16で約8 GBのVRAM。
- 検証可能性：判断に使った画像のSHA-256、各問の確率、真値軌跡、ROS bag を試行ごとに保存し、第三者が判断と行動の因果を再計算できる。

7. 限界と今後の課題

- 物資の把持・運搬は未実装。Nav2 による動的経路計画ではなく、既知配置のウェイポイント追従である。
- 自己位置は車輪オドメトリのみの推測航法で、長い経路（医療物資→避難所）では真値と最大0.24〜0.27 m ずれた（1試行が監査上限0.25 mを超過）。LiDARによる自己位置推定（AMCL等）が次の改善点。
- 評価は単純な幾何形状の小道具・3シーン・固定照明の反復で、実写画像や多様な物資への汎化は未検証。閾値0.50は確率の較正ではない。
- 2モデル合議（SmolVLM2）は本条件では全許可ケースを誤拒否し実用にならなかった（否定的結果として報告）。より大きなモデルや微調整、実機（SO-101 等）での Sim-to-Real が次の課題。
- ROSグラフ監視は認証機構ではなく、実機の安全認証を意味しない。

8. 提出動画の構成

全 2:22。3-D映像はすべて Gazebo の実レンダリング（俯瞰カメラ＋ロボット搭載カメラ）で、モック映像や模式図は使っていない。字幕・確率バー・軌跡はその試行のログ（mission.json・真値）から自動合成。走行区間は2〜3倍速、AI判断区間は等速で、速度は画面左上に常時表示している。

時刻	内容	備考
0:00–0:06	RescueMind	
0:06–0:15	なぜ「確かめる」のか	
0:15–0:24	システム構成	
0:24–1:00	デモ1：正常配置	走行2倍速・AI判断は等速（run 0a24b9d26d3949b4）
1:00–1:22	デモ2：物資の取り違え	走行3倍速・AI判断は等速（run afb3decaf41a462d）
1:22–1:45	デモ3：欠品（棚が空）	走行3倍速・AI判断は等速（run 5e343d3e6f274ea0）

時刻	内容	備考
1:45–1:53	VLMの使い方を変えて精度 37/48 → 48/48	
1:53–2:05	事前登録30試行：VLM判断 30/30・誤許可 0件	
2:05–2:15	動かして初めて分かったこと	
2:15–2:22	RescueMind	

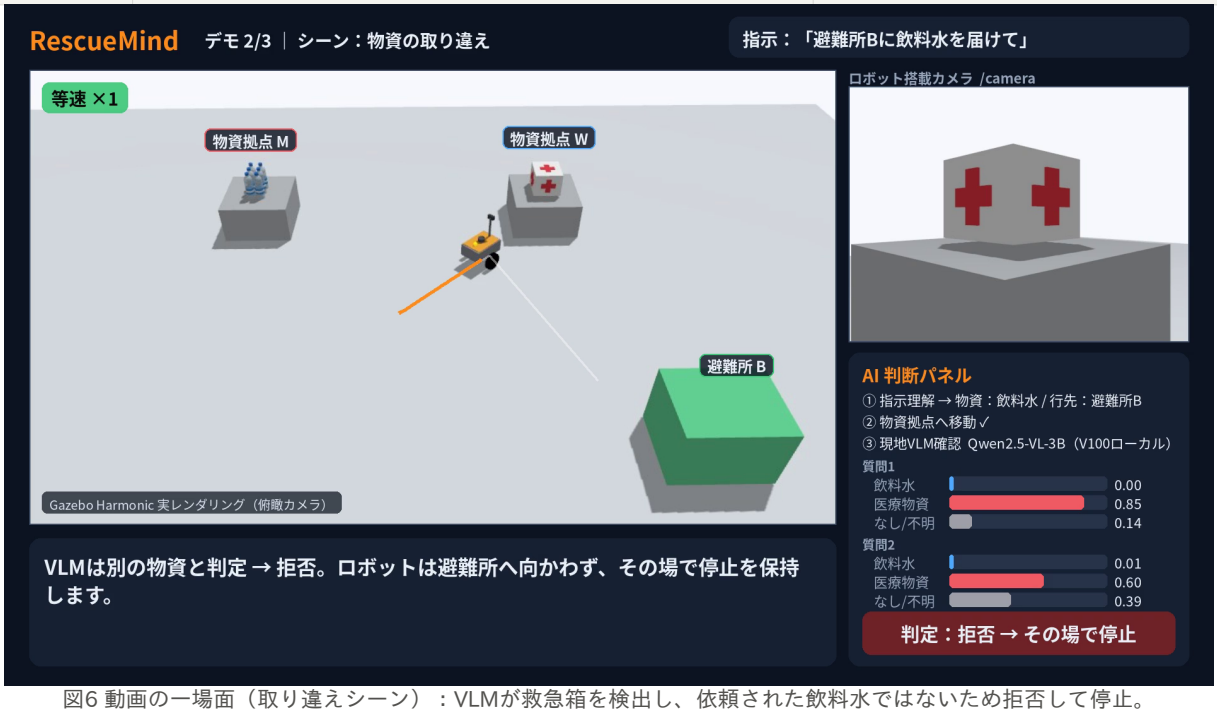


図6 動画の一場面（取り違えシーン）：VLMが救急箱を検出し、依頼された飲料水ではないため拒否して停止。

9. 再現方法

bash docker/build.sh → bash docker/run.sh python3 scripts/run_campaign.py。詳細は README（モデル重みは Hugging Face から事前取得、実行時は HF_HUB_OFFLINE=1）。テストは python3 -m unittest discover -s tests（ROS/Gazebo 不要の部分を含む）。

本資料・動画・コードに氏名・所属・メールアドレス等の個人情報は含めていない。作品は講座サンプル（差動二輪ROSシミュレーション等）を出発点に、現地VLM確認・独立安全ガード・真値監査・事前登録評価を独自に設計・実装したものである。